

VELOX LOGISTICS

TECHNOLOGY PLATFORM HANDBOOK

Document KB-007 | Version 2.3

Status: ACTIVE | Classification: CONFIDENTIAL

Effective: 2026-05-01

INTERNAL TECHNOLOGY STANDARD

Architecture, reliability, integration, and platform engineering guidance.

Synthetic enterprise document created exclusively for NexusDocs AI development and RAG evaluation.

Document Control

Document ID	KB-007	Version	2.3
Status	ACTIVE	Classification	CONFIDENTIAL
Owner	Technology Department	Effective Date	2026-05-01

Architecture Summary

The platform combines Java 21/Spring Boot transactional services, Python AI and analytics workloads, React/TypeScript interfaces, PostgreSQL, Redis, Kafka, Docker, Kubernetes, OCI, OpenTelemetry, Prometheus, and Grafana.

1. Purpose

This handbook describes the principal architecture, technology choices, service responsibilities, integration patterns, reliability controls, security practices, and observability standards used by the fictional Velox Logistics technology platform.

2. Architecture Overview

The platform uses a distributed service-oriented architecture. Customer-facing applications communicate with backend APIs, while operational workflows combine synchronous HTTP communication with asynchronous event processing. PostgreSQL is the primary transactional data store, Redis supports bounded caching and ephemeral coordination, and Apache Kafka carries domain and integration events.

3. Core Technology Stack

Backend: Java 21 with Spring Boot for core transactional and logistics services; Python for AI, analytics, document processing, and specialized automation services.

Frontend: TypeScript and React.

Data: PostgreSQL as the principal transactional source of truth; Redis for caching and short-lived state; OCI Object Storage for documents and large objects.

Messaging: Apache Kafka.

Runtime: Docker containers orchestrated by Kubernetes on Oracle Cloud Infrastructure (OCI).

Observability: OpenTelemetry, Prometheus, and Grafana.

4. Service Boundaries

The platform separates responsibilities into bounded services. Representative services include Shipment Service, Tracking Service, Claims Service, Customer Service API, Notification Service, Identity and Access components, and specialized Python-based AI/analytics workloads. A service owns its business rules and persistence boundaries unless an explicitly documented integration requires otherwise.

5. Shipment Service

The Shipment Service manages shipment creation, service selection, routing-related shipment metadata, and shipment lifecycle information that does not belong exclusively to tracking. It is implemented in Java 21 and Spring Boot and persists authoritative transactional data in PostgreSQL.

6. Tracking Service

The Tracking Service receives, validates, stores, and exposes shipment tracking events. Tracking updates may arrive from operational systems through APIs or Kafka. PostgreSQL retains authoritative tracking records. Redis may cache frequently requested tracking views, but **Redis is never the source of truth for tracking history**.

7. Claims Service

The Claims Service manages claim cases, statuses, evidence references, investigation workflow, and resolution metadata. It integrates with shipment and tracking capabilities without directly modifying data owned by those services. Cross-service workflows should favor stable APIs or events rather than shared database tables.

8. Synchronous Communication

HTTP APIs are used when a caller requires an immediate response or when the operation naturally represents a request/response interaction. Synchronous dependencies must use explicit timeouts. Retries are permitted only when the operation is safe to retry and should use bounded retry policies with backoff.

9. Event-Driven Communication

Apache Kafka is used for asynchronous workflows and domain/integration events such as tracking updates, shipment lifecycle changes, notification triggers, and analytics feeds. Event-driven communication reduces temporal coupling and allows consumers to process events independently. Kafka is not used merely to replace every HTTP call; the integration pattern must match the business requirement.

10. Event Contracts

Events should include an event identifier, event type, schema version, occurrence timestamp, producer, relevant aggregate identifier, and payload. Consumers must not assume that delivery guarantees alone prevent duplicate business processing. Event schemas must evolve compatibly or through an explicit versioning strategy.

11. Idempotency

Consumers handling events that can be redelivered must implement idempotent processing. A representative strategy stores the unique event identifier or business idempotency key in durable storage as part of the processing boundary. If the identifier has already been successfully processed, the consumer must avoid applying the business effect a second time. **Kafka delivery and retries do not remove the need for application-level idempotency.**

12. Retries and Dead-Letter Handling

Transient failures may be retried using bounded attempts and exponential or otherwise controlled backoff. A consumer that repeatedly fails to process an event must not retry forever. After the configured retry policy is exhausted, the event may be routed to a **dead-letter topic (DLQ)** together with diagnostic context. DLQ events require monitoring and an explicit replay or remediation procedure.

13. PostgreSQL

PostgreSQL is the primary transactional database for authoritative business records. Services should own their schemas or logical persistence boundaries. Direct cross-service table access is discouraged because it creates hidden coupling and bypasses service-level business rules.

14. Redis and Cache Policy

Redis is used for data that benefits from low-latency access and can tolerate bounded staleness, such as selected tracking projections or reference data. Cache entries must have an appropriate expiration or invalidation strategy. A cache miss must be recoverable from an authoritative system. Critical business data must not exist only in Redis.

15. Cache Consistency

When authoritative data changes, the responsible service should invalidate or refresh affected cache entries according to the use case. The system must tolerate stale or missing cache entries within documented limits. Cache correctness

must never depend on Redis being permanently available.

16. Containerization

Services are packaged as Docker containers using reproducible builds. Runtime images should minimize unnecessary packages, avoid embedding credentials, run with the least privileges practical, and expose health information needed by orchestration.

17. Kubernetes

Kubernetes manages service deployment, scaling, service discovery, configuration integration, and workload recovery. Readiness probes determine whether a workload should receive traffic; liveness probes help detect workloads that require restart. Applications must not treat a successful container start as proof that all dependencies are healthy.

18. Oracle Cloud Infrastructure

Production-like workloads are hosted on OCI. The architecture may use OCI Compute and Kubernetes infrastructure, networking controls, load balancing, Object Storage, managed database capabilities where appropriate, and centralized secrets/configuration mechanisms. Infrastructure definitions should be automated and reviewable rather than maintained solely through manual console changes.

19. Resilience

Services must assume that networks and dependencies can fail. Controls include timeouts, bounded retries, backoff, idempotency, graceful degradation, health checks, asynchronous decoupling, and operational alerting. Circuit-breaking or similar protection may be used where repeated downstream failures could cause cascading resource exhaustion.

20. Observability

OpenTelemetry provides standardized instrumentation for traces and telemetry correlation. Prometheus collects operational metrics, and Grafana provides dashboards and visualization. Logs should be structured and include correlation identifiers where practical. Metrics should cover latency, traffic, errors, saturation, consumer lag, retry activity, DLQ activity, and critical business-process signals.

21. Distributed Tracing

Requests crossing service boundaries should propagate trace context where supported. Event producers should include correlation metadata that allows asynchronous processing to be associated with relevant business flows. Tracing complements logs and metrics; it does not replace them.

22. Security

Authentication and authorization must be enforced at appropriate boundaries. Secrets must not be stored in source code or container images. Service credentials should follow least-privilege principles. Sensitive data must be protected in transit and at rest using platform-supported controls. Access to confidential internal documentation and operational systems must be auditable.

23. API Security

Externally reachable APIs must validate inputs, apply authorization rules, and avoid exposing internal implementation details through error responses. Rate limiting or abuse controls should be applied where risk justifies them. Security-relevant events should be logged without exposing secrets.

24. Data Ownership

Each service is responsible for the integrity of the business data it owns. Other services should consume that data through defined APIs, events, or approved replicated projections. Shared databases must not become an informal integration mechanism.

25. Failure Scenario: Duplicate Tracking Event

If a tracking event is delivered more than once, the Tracking Service consumer evaluates its event identifier or idempotency key. A previously completed event must not create a second tracking transition or duplicate downstream side effect. The duplicate may be recorded for observability without reapplying the business operation.

26. Failure Scenario: Consumer Repeatedly Fails

If a Kafka consumer encounters a transient error, the configured retry policy may attempt processing again. If the event continues to fail after the bounded retry policy is exhausted, it is routed to the appropriate DLQ. Monitoring should alert the responsible team, which can inspect the diagnostic context, correct the underlying issue, and use an authorized replay procedure.

27. Failure Scenario: Redis Unavailable

If Redis is unavailable, services should fall back to the authoritative data source when the operation permits it, accepting increased latency where necessary. A Redis outage must not cause loss of authoritative tracking or shipment data because those records are persisted in PostgreSQL.

28. Architecture Decision Principles

Technology choices should optimize for explicit ownership, evolvability, reliability, operational visibility, and business requirements rather than novelty. Synchronous calls are preferred for immediate request/response needs; asynchronous events are preferred when temporal decoupling, fan-out, or independent processing provides material value.

29. Relationship to NexusDocs AI

The Technology Platform Handbook is an internal knowledge source that NexusDocs AI can retrieve when answering architecture and engineering questions. Its CONFIDENTIAL classification is intentionally different from public FAQ material so future versions of NexusDocs can demonstrate metadata-aware authorization and retrieval controls.

30. Revision History

Version	Effective date	Status	Summary
2.3	2026-05-01	ACTIVE	Service boundaries, Kafka, idempotency, retries/DLQ, cache policy, OCI/Kubernetes, security, resilience, and observability.

Synthetic data notice: Velox Logistics is fictional. This document exists exclusively for NexusDocs AI technical retrieval testing, architecture reasoning, access-control experiments, and RAG evaluation.